

从 AI 助手到 AI 团队

多 Agent 协作如何改变科学计算

AI for Science · Codex 科研工作流实战

计算机学院 苏悦馨

Codex 与对话式 AI 差异

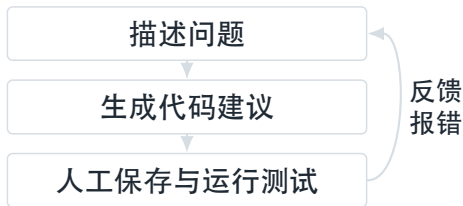


对话 AI 生成局部文本代码；
Codex 直接对项目目录与文件进行读写和修改

对话式 AI

单次交互模式

- 解释概念、算法与错误信息
- 生成独立函数或脚本片段
- 修改输入的局部代码
- 依据人工反馈提供补充建议



代码的保存、环境运行与结果
验证需人工执行

Codex—项目环境访问

```
research-project/  
+-- AGENTS.md  
+-- TASK.md  
+-- data/  
    +-- experiment.csv  
+-- analysis.py  
+-- tests/  
    +-- test_acceptance.py  
+-- artifacts/
```

项目环境访问

直接读取配置规则、已有代码、数据文件与测试文件，在当前约束与既有代码库基础上继续开发

操作对象为完整项目状态

Codex—执行代码

对源码文件进行修改

```
analysis.py
- scale = pixels / 255.0
+ scale = pixels / 16.0
- X = df.values
+ X = df[pixel_cols].values
+ seed = 42
```

通过终端执行与验证

```
$ python analysis.py
loading data/experiment.csv
split fixed: seed=42
writing
artifacts/metrics.json
writing artifacts/report.md
$ pytest -q
3 passed in 1.84s
```

调整源码文件，并通过终端获取程序运行结果

Codex—任务交付

任务契约与交付规范

输入 `data/experiment.csv`

步骤 清洗 · 统计 · 制图

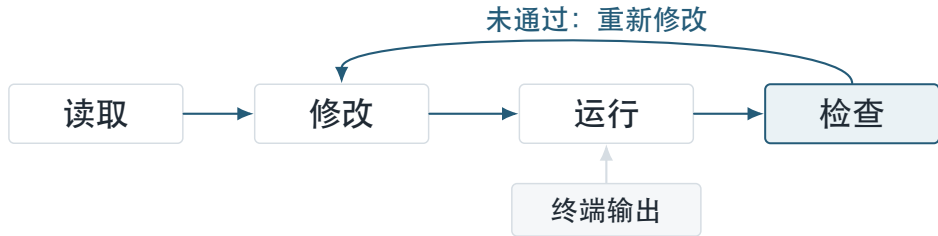
交付 图表 · 指标 · 报告

验证 命令可复现，测试通过

```
artifacts/  
+-- figure.png  
+-- metrics.json  
+-- report.md  
+-- reproduce.log  
acceptance: PASS
```

最终产物包括可重复运行的代码、测试结果与数据报表

Codex—闭环执行



闭环执行流程：系统依据实际运行输出判断状态，并自动决定后续修复

Codex—适用任务

适用任务类型

数据处理

检查缺失值与数据范围；完成清洗、合并与格式转换

例：统一多批实验记录字段

批量计算

编写计算逻辑、配置参数扫描并记录运行日志

例：比较不同正则化参数

验证与汇总

执行测试套件、绘制结果图表并导出指标数据

例：核对误差曲线与输出文件

未明确定义目标、缺乏判断标准或计算逻辑
不固定的开放式问题不适合由 Codex 处理

Codex—职责边界

职责分工边界

Codex 执行范围

实现 · 运行 · 校验

按规则修改代码

执行计算并记录日志

导出图表与测试脚本

研究者决策范围

目标 · 假设 · 解释

确定研究问题与计算口径

设定模型假设与关键参数

评估物理含义与结论边界

Codex 负责满足明确约束条件的编程实现；
研究者负责研究方案与科学逻辑的确定

Codex Live Demo

MNIST 手写数字识别：从原始 CSV 到结果图

MNIST 数据

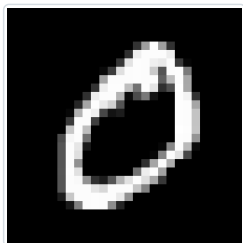
0, 0, 0, 0, 0, ..., 255, ...

第 1 项

数字标签

后 784 项

按行排列的灰度像素



$784 = 28 \times 28$

像素范围 0 到 255

60,000 训练行

10,000 测试行

无表头

每行 785 个整数

工作目录

```
example/  
+-- README.md  
+-- AGENTS.md  
+-- TASK.md  REQUEST.md  
+-- data/raw/  
+-- expected/  
+-- src/  tests/  outputs/  
+-- .agents/skills/  
+-- .codex/agents/
```

读入

AGENTS.md 修改范围与文件位置

TASK.md 实验要求

REQUEST.md 图表与报告要求

data/README.md CSV 格式

expected/README.md 验收标准

输出

src/ 与 tests/ 代码与测试

outputs/ 与 REPORT.md 结果与报告

启动指令

项目地址: <https://github.com/Macondooo/codex-research-examples/tree/main/MNIST>

```
Implement and run this MNIST experiment.  
Verify the outputs and summarize the results.
```

AGENTS.md

规定读取顺序、修改范围和文件位置。

TASK.md

定义模型、指标、测试和输出。

REQUEST.md

要求图表适合投影，报告解释分数和常见混淆。

工程化进阶——Codex 使用技巧

从一次任务执行，走向可长期使用的科研工程方法

多层上下文

记得住：重要信息放哪里？

上下文定义

上下文是 **Agent** 当前能够读取的事实与规则

当前任务

本次任务内容
输入、交付与验收

TASK.md / Prompt

项目规则

长期执行规范
边界、入口与约束

AGENTS.md + docs/

当前事实

文件当前状态
终端、测试与日志

files + terminal

将稳定信息写入项目文件，供 Agent 持续读取

如何管理项目上下文

根据上下文的有效期分类



Prompt 保存临时要求；项目规则与重复流程写入项目文件

如何管理项目上下文

按上下文的作用范围确定位置

```
project/  
+-- AGENTS.md  
+-- modeling/  
    +-- AGENTS.md  
+-- visualization/  
    +-- AGENTS.md  
+-- docs/workflow.md  
+-- tests/
```

根目录 **Markdown**

覆盖整个仓库：数据边界、API 兼容性、测试要求

子目录 **Markdown**

约束局部目录：Python 环境、Rust 编码、格式规范

Workflow Markdown

记录重复步骤：检查、运行、审计、发布

上下文管理 Demo

Lance

github.com/lancedb/lance

Lance 是面向多模态 AI 数据集的开源 Lakehouse 与列式格式项目，支持高性能随机访问、向量检索、版本化写入和数据生态集成

仓库包含 Rust 核心库、Python 与 Java binding、protobuf schema、格式规范、测试数据和发布流程

Agent 沿目录读取长期规则、局部规则和可重复 workflow

上下文管理 Demo

Lance 分开存放公共规则、目录规则和工作流文档

```
lance/  
+-- AGENTS.md  
+-- CONTRIBUTING.md  
+-- release_process.md  
+-- rust/AGENTS.md  
+-- python/AGENTS.md  
+-- java/AGENTS.md  
+-- protos/AGENTS.md
```

AGENTS.md

统一项目架构、命令、编码、测试、PR 与 review 规则

rust / python / java / protos AGENTS.md

定义语言环境、构建命令、API 设计和格式治理

AGENTS.md

Coding Standards

General

- Always use English in code, examples, and comments.
- Code is for readability, not just execution. Only add meaningful comments and tests.
- Comments should explain non-obvious "why" reasoning, not restate what the code does.
- Remove debug prints (`println!`, `dbg!`, `print()`) before merging — use `tracing` or logging frameworks.
- Think carefully before adding a helper: only introduce one when it materially reduces cognitive load or eliminates substantial duplication, and do not add thin wrappers that only rename or forward existing calls.
- Keep PRs focused — no drive-by refactors, reformatting, or cosmetic changes.
- Be mindful of memory use: avoid collecting streams of `RecordBatch` into memory; use `RoaringBitmap` instead of `HashSet<u32>`.

根 AGENTS.md 统一规定代码语言、可读性、注释、调试输出与内存使用

AGENTS.md

Testing Standards

- **All bugfixes and features must have corresponding tests. We do not merge code without tests.**
- Keep local unit tests lightweight: each test case should finish within one second on typical developer hardware. Split independent parameter matrices and use the smallest fixture or model that preserves the asserted behavior; do not relax assertions, coverage, or recall thresholds to meet the budget.
- Use `rstest` (Rust) or `@pytest.mark.parametrize` (Python) for tests that differ only in inputs. Use `#[case::{name}(...)]` for readable case names.
- Replace `print()` in tests with `assert` — prints don't catch regressions.
- Extend existing tests instead of adding overlapping new ones. Add to existing test files.
- Link a GitHub issue when skipping a test — never bare `@pytest.mark.skip` or `@Ignore` without a tracking URL.
- Include multi-fragment scenarios for dataset operations (reads, indexes, scans).
- Cover NULL edge cases in index tests: null items, all-null collections, empty collections, null columns.
- Vector index tests must assert recall metrics (≥ 0.5 threshold), not just verify creation succeeds.

根 AGENTS.md 要求功能变更配套测试，并约束测试效率、覆盖与召回指标

python/AGENTS.md

Commands

- Environment: use `uv` for all local Python environment setup in this repository.
- First step in every new worktree or fresh checkout: run `make install` from `python/` before any Python command. This runs `uv sync` (dev and test dependencies are included by default via `[tool.uv] default-groups`) and sets up pre-commit hooks. Add `--group benchmarks`, `--extra torch`, or `--extra geo` only when needed.
- `uv sync` builds the local `pylance` Rust extension as part of environment setup. This can take a long time. Start it early, let it finish, and do not interrupt it or switch to a different setup path just because the build is slow.
- Only run `uv sync` again when dependencies change (e.g., after pulling new commits that update `pyproject.toml` or `uv.lock`).
- Command execution: always use `uv run ...` for Python-related repository commands. Do not rely on a globally activated environment.
- Never invoke bare `python`, `pytest`, `pip`, `maturin`, `make test`, `make doctest`, `make lint`, or `make format` for repository work.

python/AGENTS.md 指定 uv 环境，以及安装、构建、测试和 lint 的统一入口

python/AGENTS.md

API Design

- Keep bindings as thin wrappers — centralize validation and logic in Rust core.
- Extend existing methods with named arguments instead of adding new methods that accept policy/config objects — the Python API should feel Pythonic (e.g., `cleanup_old_versions(..., retain_versions=N)`), not mirror Rust builder patterns.
- Pass all fields to Python dataclass constructors via PyO3, converting Rust `None` to `py.None()` instead of omitting args — dataclass constructors require all positional params.
- Use parameterized type hints (e.g., `list[DatasetBasePath]`, `Optional[Dict[str, str]]`) — never bare generics. Keep docstring type descriptions in sync with hints.

Testing

- Add tests to existing `test_{module}.py` files rather than creating new test files for the same module.

python/AGENTS.md 约束 binding 设计、类型标注和测试文件组织

Agents 任务拆分

分得开：谁来做，在哪里改？

主 Agent 和 Subagent

主 Agent 先划分子任务边界，再分发执行
Subagent 执行边界明确的子任务

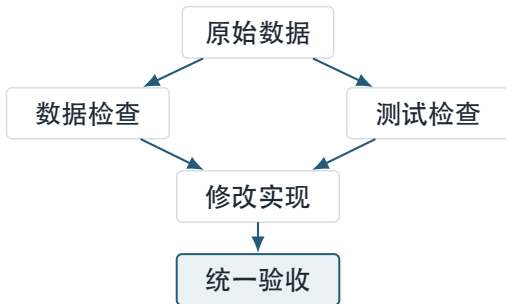


如何拆分 subagent

先确认任务依赖，再安排并行执行

可并行：没有前后
依赖的只读检查

需等待：依赖上游
输出的实现与验收



输入边界清楚、输出格式明确，且不会同时写入同一批文件

如何拆分 subagent

每个 **Subagent** 接收一份边界明确、结果可验收的任务

```
task: data-audit
input: data/assay.csv
write: artifacts/audit.md only
output: missing/range report
done: required fields checked
```

输入 能读取什么

写入范围 能修改哪些文件

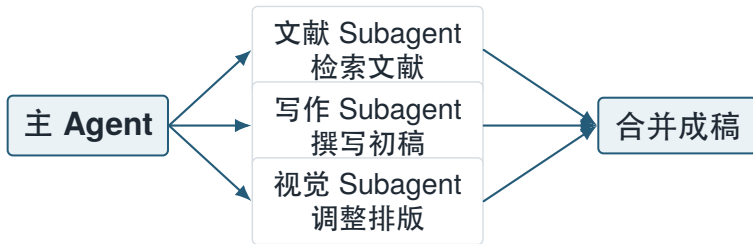
输出格式 如何汇总

完成条件 何时完成

窄范围与明确的验收条件有助于稳定汇总结果

Example

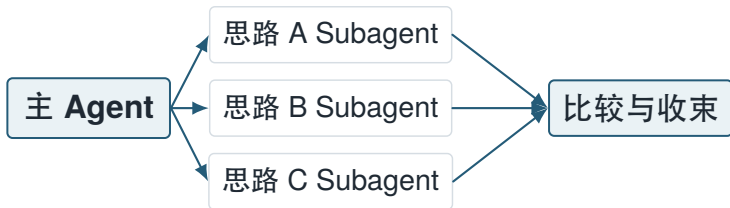
论文写作：不同 **Subagent** 分别检索文献、撰写初稿和调整排版



主 Agent 汇总成稿，并检查引用、论证与图文一致性

Example

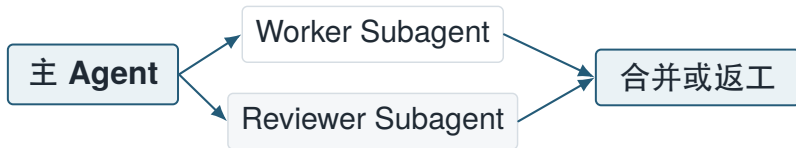
数学推导：不同 **Subagent** 独立推进候选思路



比较阶段检查假设、推导步骤、反例与未解决卡点

Example

代码任务：**Worker** 负责实现，**Reviewer** 负责审计



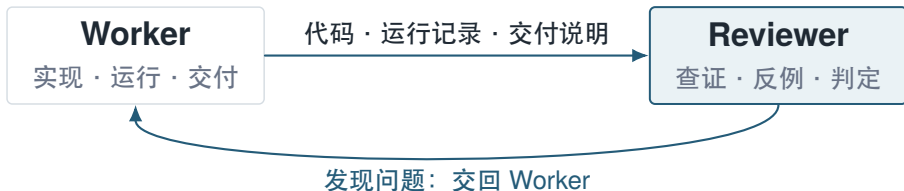
Reviewer 不直接修改代码，避免实现与验收由同一角色完成

Worker 与 Reviewer

生成与验收分离

Worker 与 Reviewer 的关系

同一任务中，实现与验收应由不同角色负责。



Worker 不自评自过，**Reviewer** 不参与实现。

如何构建 Reviewer

将正确性标准写成 Reviewer 可读取、可执行的验收材料。

review/acceptance.md

人可读的验收规则

科学口径与结论边界

文本与报告要求

异常情况与处理原则

tests/

机器可执行的验收规则

函数行为

schema 与数值条件

文件、命令与返回状态

先固定验收标准，再执行检查；Reviewer 不临场改变口径。

Worker-Reviewer Demo

Iris 分类任务： 演示实现、交付与独立复核。

数据与设置

数据集 scikit-learn 内置 Iris
划分 固定 random_state=42
处理 特征标准化
模型 KNN ($k = 5$)

输出与复核

输出 准确率与混淆矩阵
可视化 带标签的混淆矩阵热图
测试 覆盖数据处理与预测流程
复核 核对代码、测试与输出一致性

代码仓库

[https://github.com/Macondooo/codex-research-examples/
tree/main/worker_reviewer](https://github.com/Macondooo/codex-research-examples/tree/main/worker_reviewer)

Skills 与 MCP

固定流程与外部工具

Skills

重复出现

同类任务反复出现时，读取范围、检查方法和交付要求通常不变。

保存流程

Skill 把这些约定保存在项目中。Codex 根据 `description` 判断是否调用，再读取具体步骤。

01

收到请求

02

匹配 **description**

03

读取 **SKILL.md**

需要指定某个 Skill 时，在请求中写 `$skill-name`。

Skill 目录

```
my-skill/  
+-- SKILL.md  
+-- references/  
+-- scripts/  
+-- assets/
```

SKILL.md 调用条件与执行步骤

references/ 指南、术语表与示例

scripts/ 检查与转换脚本

assets/ 模板与固定资源

写作 Skills Demo

Master-cai/Research-Paper-Writing-Skills

GitHub

Skill entry

SKILL.md
agents/
openai.yaml

references/

abstract.md
introduction.md
related-work.md
method.md
experiments.md
conclusion.md
paper-review.md

这是一个完整的 Skill 目录。SKILL.md 负责入口；references/ 按论文章节拆分规则；agents/ 保存辅助配置。

写作 Skills Demo

research-paper-writing/SKILL.md

GitHub

name: research-paper-writing

description: Improve academic paper writing quality for ML/CV/NLP-style papers with clear section structure, paragraph flow, and reviewer-facing presentation.

Overview

Use this skill to rewrite a research paper into a reviewer-friendly, high-clarity draft. Prioritize logical flow and evidence-backed claims.

Core Workflow

1. Clarify the paper story before sentence-level edits.
2. Use section-specific guidance in references/.

SKILL.md 先说明用途和触发条件，再给出全局写作流程。需要章节细则时，继续读取 references/。

写作 Skills Demo

research-paper-writing/references/related-work.md

GitHub

Goal

Position your work against the most relevant lines of research, and make your novelty easy to verify.

Workflow

1. List directly competing and recent baseline papers first.
2. Group literature by technical topic, not by publication year alone.
3. For each topic, summarize the common paradigm and the limitation relevant to your challenge.
4. End each topic by clarifying your distinction.

章节规则把 Related Work 的写作步骤单独列出：先找直接相关工作，再按技术主题组织，并说明本文与已有工作的差异。

MCP

MCP 为 **Codex** 提供项目外的工具和上下文。

MCP server 连接文档、数据库或服务，并返回查询结果。



Context7

Context7 通过 **MCP** 提供软件库文档查询。

先确定软件库，再按问题检索文档和代码示例。

接入 **Codex**

```
codex mcp add context7 -- \  
  npx -y @upstash/context7-mcp  
  
codex mcp list
```

01 resolve-library-id

确定软件库与版本

02 query-docs

检索与问题相关的文档片段

两次调用

01 resolve-library-id

```
libraryName: scikit-learn  
query: fit scaling only on  
       training data
```

返回 library ID、版本和文档覆盖情况。

02 query-docs

```
libraryId: /scikit-learn/scikit-learn  
query: fit scaling only on  
       training data
```

返回相关文档片段和代码示例。

请求示例：请用 Context7 核对 scikit-learn 中 StandardScaler 与数据划分的正确顺序。

返回内容

查询结果包含库标识、文档出处和相关代码片段。

库标识

/scikit-learn/
scikit-learn

文档出处

doc/modules/sgd.rst

query-docs 节选

```
scaler = StandardScaler()
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)

# Or use a pipeline
est = make_pipeline(
    StandardScaler(), SGDClassifier()
)
```

代码核对

原写法

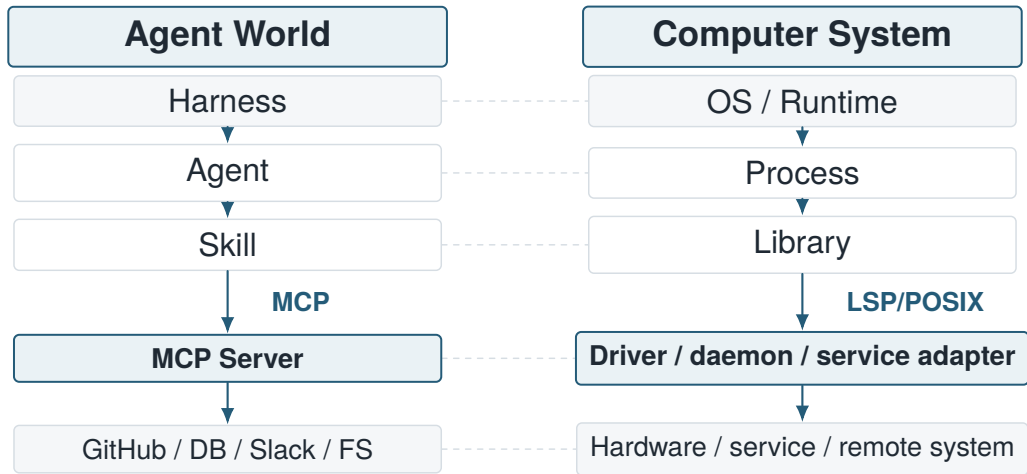
```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)  
X_train, X_test, y_train, y_test =  
    train_test_split(X_scaled, y)
```

核对后

```
X_train, X_test, y_train, y_test =  
    train_test_split(X, y)  
model = make_pipeline(  
    StandardScaler(),  
    KNeighborsClassifier(5)  
)  
model.fit(X_train, y_train)
```

StandardScaler 只在训练数据上拟合，测试数据只做 transform。

Summary



Codex 个人使用经验

Codex 实现代码

简单任务耗时过长

按任务难度匹配模型和推理强度

Extra High 模式反复校验哈希

仅在确有完整性校验需求时检查 hash

为验证正确性生成大量测试代码

在 prompt 中明确验证范围与停止条件

项目变大后难以追踪改动

每次修改同步维护人类可读的 Markdown

Skill 过多会干扰工具选择

在 prompt 中明确允许或要求使用的 skills

加载了工具读取文件

自定义技能要求我先基线检查再编辑。现在读取现有 `main.tex` 和两份附件，记录第一章源代码哈希；后续只在 `\end{document}` 前追加 `\input`，并用哈希确认原 9 页正文一字未改。

tests

- test_compare.py
- test_config.py
- test_data.py
- test_end_to_end.py
- test_generators.py
- test_model.py
- test_position.py
- test_training_utils.py

Codex 写文章、做 PPT

语言机械、生硬

用写作 skill 固定风格，并提供同领域文本作为范例；必要时再润色。

文章逻辑松散

先写清逻辑大纲，再由 Codex 展开正文。

PPT 排版混乱

明确版式要求，调用视觉模型，并将稳定规范整理为 skill。

风格目标

行文应接近 `ch_ref/` 中项目申请书正文的学术中文风格：背景铺垫充分，逻辑推进自然，术语准确，语气克制。避免机械地重复“这个 theorem”“该 proposition 的意义在于”“表明了一个共同模式”等模板化句式。

段落组织

- 先交代研究对象和必要性，再说明已有方法的适用范围和不足，最后自然引出当前章节的技术问题。
- 常用衔接包括“总体来看”“进一步地”“与此同时”“由此可见”“关键在于”“因此，一个自然的问题是”，但不要堆叠使用。
- formal statement 后的解释应从参数、适用条件、与基线的关系三方面展开，不把解释写成条目清单。
- 综述章节避免“本文提出”“本人工作”等表述；本人已有成果只放在第四章。

句式偏好

- 用完整中文句子承接公式：公式前说明要刻画什么，公式后解释参数和适用范围。
- 保留必要英文术语，如 oracle、regret、cut fraction、LP relaxation，但中文谓语和逻辑连接要自然。
- 避免泛化判断，例如“非常重要”“具有重大意义”应落到具体技术原因上。
- 避免过度口语和过度机械：少用“这个结果的意义在于”，可改为“这一结论的作用在于”“该界限说明”“由该式可见”。

模板实操——亲手跑通一段科研任务

目录结构

```
experiment/  
+-- README.md  
+-- AGENTS.md  
+-- TASK.md  
+-- REQUEST.md  
+-- data/  
|   +-- README.md  
+-- expected/  
    +-- README.md
```

预置文件

定义固定任务、仓库规则、数据说明和验收标准。

本次填写

REQUEST.md 是唯一需要编辑的文件，用来补充本次实验要求。

README.md

使用说明

列出阅读、填写、执行和检查的顺序。

1. Read TASK.md and data/README.md.
2. Complete REQUEST.md.
3. Open this directory in Codex.
4. Check expected/README.md.

打开工作目录后，按这四步完成实验。

AGENTS.md

仓库指令

规定 Codex 在当前目录中的读取顺序、修改范围和验证要求。

```
Read TASK.md, REQUEST.md, and data/README.md.  
Keep the supplied material unchanged.  
Put code in src/, tests in tests/, outputs in  
outputs/.  
  
Run tests and analysis before reporting  
completion.
```

同时约定代码、测试和输出的存放位置，避免改动原始数据。

TASK.md

任务书

写明实验目标、必做项、交付物和范围。

Outcome

实验目标

Behavior

必须完成

Deliverables

交付内容

Boundaries

范围限制

固定要求写在 TASK.md；REQUEST.md 只补充本次选择。

data/README.md

数据说明

说明输入文件、字段含义和处理规则。

寿命预测

字段说明
预测目标
已知缺失值

词频统计

语料文件
分词用例
分词规则

填写 REQUEST.md 前，先确认字段名、文件结构和处理约定。

expected/README.md

验收标准

列出完成后要核对的文件、测试和结果。

文件完整

核对路径与内容

测试通过

运行自动测试

结果一致

核对指标、
表格与图表

它规定验收证据，不限定内部实现。

REQUEST.md

```
# Request
```

```
## Outcome
```

```
## Additional Constraints
```

```
## Acceptance Criteria
```

Outcome

写明本次要交付的最终结果。

Additional Constraints

补充模型、工具、可视化或报告格式；没有可留空。

Acceptance Criteria

写明验收所需的文件、命令、测试或指标。

只填写 TASK.md 尚未规定的要求，不重复固定任务。

寿命预测

```
data_analysis/  
+-- TASK.md  
+-- REQUEST.md  
+-- data/life_expectancy.csv  
+-- data/README.md  
+-- expected/README.md
```

核对数据

你会拿到 32 条社区记录，里面有收入、医疗和教育等指标；预测目标是平均寿命。

完成比较

用一个回归模型完成预测，并与均值基线比较。

保存结果

代码、测试、结果文件和 REPORT.md。

请求示例

```
# Request
## Outcome
Complete the life-expectancy regression project in TASK.md.
## Additional Constraints
Use Ridge regression in a scikit-learn Pipeline. Use pytest.
## Acceptance Criteria
The documented analysis and test commands complete successfully.
```

这个例子只补充模型和实现方式；TASK.md 已有内容不再重复。

词频统计

```
text_analysis/  
+-- TASK.md  
+-- REQUEST.md  
+-- data/corpus.csv  
+-- data/tokenizer_cases.csv  
+-- data/README.md  
+-- expected/README.md
```

确认差异

同一句英文，分词规则一变，词频结果也会变化。

按规则统计

按目录中的规则处理 20 条文本，并用 6 组样例检查分词。

保存结果

代码、测试、词频表、统计摘要、图表和 REPORT.md。

请求示例

```
# Request
## Outcome
Complete the tokenization and word-frequency project in TASK.md.
## Additional Constraints
Use regular expressions and the Python standard library.
## Acceptance Criteria
The documented analysis and test commands complete successfully.
```

这个例子只补充分词实现和依赖范围；TASK.md 已有内容不再重复。

Hands-on: Build Your Own Codex Workflow

项目地址：

[https://github.com/Macondooo/
codex-research-examples/tree/main/
code_data_text_analysis](https://github.com/Macondooo/codex-research-examples/tree/main/code_data_text_analysis)

| data_analysis/text_analysis 任选一个，
| 完成 REQUEST.md，并在 codex 上运行。

Q&A

Q&A

1 上下文窗口经常过长很耗 token，主要靠 prompt 和 skill 来指引 LLM 输出，不知道是否有更高效的办法？

2 无法解决 gpt 防御式写作严重的问题